

MongoShell command

Projection (sirf kuch fields dikhani ho)

Name aur phone hi chahiye

JavaScript

```
db.users.find(  
  { isActive: true },  
  { name: 1, phone: 1, _id: 0 }  
)
```

JavaScript

```
db.users.findOne({ name: "Tarun Patidar" })
```

```
db.users.find({ age: 22 })
```

```
db.users.find({ "address.city": "Indore" })
```

```
db.users.find({ skills: "MongoDB" })
```

```
db.users.find({}, { name: 1, phone: 1, _id: 0 })
```

2) countDocuments()

Total documents count

JavaScript

```
db.students.countDocuments()
```

Condition ke saath count

JavaScript

```
db.students.countDocuments({ city: "Indore" })
```

Example

JavaScript

```
db.employees.countDocuments({ department: "IT" })
```



JavaScript

```
db.students.find().limit(5)
```

Salary वाले first 3 employees

JavaScript

```
db.employees.find({}, { name: 1, salary: 1, _id: 0 }).limit(3)
```

4) skip()

First 5 skip karke next documents

JavaScript

```
db.students.find().skip(5)
```



Pagination style: first 10 skip, next 5

</> JavaScript

```
db.students.find().skip(10).limit(5)
```

5) sort()

Ascending sort by fees

</> JavaScript

```
db.students.find().sort({ fees: 1 })
```

Descending sort by salary

</> JavaScript

```
db.employees.find().sort({ salary: -1 })
```

Multiple field sort

</> JavaScript

```
db.employees.find().sort({ city: 1, salary: -1 })
```

6) sort + limit

Top 5 highest salary employees

</> JavaScript

```
db.employees.find({}, { name: 1, salary: 1, _id: 0 })  
.sort({ salary: -1 })  
.limit(5)
```

Lowest 3 fee students

JavaScript

```
db.students.find({}, { name: 1, fees: 1, _id: 0 })  
.sort({ fees: 1 })  
.limit(3)
```

7) Single document field read

Example: `students`, `employees`, `products` me simple fields

Students from Indore

JavaScript

```
db.students.find({ city: "Indore" })
```

11) Projection on embedded fields

Sirf title aur comments user dekhna

JavaScript

```
db.posts.find(  
  {},  
  { title: 1, "comments.user": 1, _id: 0 }  
)
```

12) Array size check

Jis post me exactly 3 tags ho

JavaScript

```
db.posts.find({ tags: { $size: 3 } })
```

76a50b8e1-ec10-83ee-929e-0780e12b4e4f

Delivered orders ka count

JavaScript

```
db.orders.countDocuments({ status: "Delivered" })
```

Latest-like top 5 posts by likes

JavaScript

```
db.posts.find({}, { title: 1, likes: 1, _id: 0 })  
.sort({ likes: -1 })  
.limit(5)
```

Highest fee courses

JavaScript

```
db.courses.find({}, { courseName: 1, fees: 1, _id: 0 })  
.sort({ fees: -1 })  
.limit(5)
```



JavaScript

```
use companyDB
```

```
db.students.countDocuments()
```

```
db.students.countDocuments({ city: "Indore" })
```

```
db.students.find().limit(5)
```

```
db.students.find().skip(5)
```

```
db.students.find().skip(10).limit(5)
```

```
db.students.find().sort({ fees: 1 })
```

```
db.employees.find().sort({ salary: -1 })
```

```
db.posts.find({ tags: "mongodb" })
```

```
db.posts.find({ "comments.user": "Rahul" })
```

```
db.posts.find({ "comments.msg": "Good" })
```

```
db.posts.find({}, { title: 1, "comments.user": 1, _id: 0 })
```

```
db.posts.find({ tags: { $size: 3 } })
```



A) Single / normal field

city = Indore

JavaScript

```
db.students.find({ city: { $eq: "Indore" } })
```

status = Delivered

JavaScript

```
db.orders.find({ status: { $eq: "Delivered" } })
```

B) Array field

`posts.tags` array hai:

</> JavaScript

```
tags: ["mongodb","nodejs","database"]
```

Jis post ke tags me `"mongodb"` ho

</> JavaScript

```
db.posts.find({ tags: { $eq: "mongodb" } })
```

C) Embedded document / nested field

Agar nested field access karna ho to **dot notation** use hota hai.

Jis comment ka msg = "Nice"

</> JavaScript

```
db.posts.find({ "comments.msg": { $eq: "Nice" } })
```

D) Embedded array (array of objects)

Jis post me user = Rahul comment ho

</> JavaScript

```
db.posts.find({ "comments.user": { $eq: "Rahul" } })
```

A) Single field

</> JavaScript

```
db.students.find({ city: { $ne: "Indore" } })
```

</> JavaScript

```
db.orders.find({ status: { $ne: "Delivered" } })
```

B) Array field

Jis post me "mongodb" tag na ho

</> JavaScript

```
db.posts.find({ tags: { $ne: "mongodb" } })
```

C) Embedded document field

Jis post me comment msg "Good" na ho

❏ JavaScript

```
db.posts.find({ "comments.msg": { $ne: "Good" } })
```

D) Embedded array

Jis post me Rahul user wala comment na ho

❏ JavaScript

```
db.posts.find({ "comments.user": { $ne: "Rahul" } })
```

A) Single field

fees > 70000

⌘ JavaScript

```
db.students.find({ fees: { $gt: 70000 } })
```

salary > 100000

⌘ JavaScript

```
db.employees.find({ salary: { $gt: 100000 } })
```

B) Array field

Jis post me `tags` array ka size compare directly `$gt` se nahi hota

Array ke size ke liye `$size` use hota hai, comparison operator नहीं.

✗ Direct नहीं:

❏ JavaScript

```
db.posts.find({ tags: { $gt: 2 } }) // meaningful array size query nahi
```

Correct idea:

❏ JavaScript

```
db.posts.find({ tags: { $size: 3 } })
```

Important: normal array values par `$eq/$ne` common hai.

Numeric array ho to `$gt/$lt` meaningful hota hai but tumhare `posts.tags` string array hai.



A) Single field

⌞ JavaScript

```
db.students.find({ age: { $gte: 24 } })
```

⌞ JavaScript

```
db.products.find({ rating: { $gte: 4 } })
```

B) Array field

String array me practical use कम है, but equality-style matching hota hai.

C) Embedded document field

likes $\geq$ 500 वाले posts

JavaScript

```
db.posts.find({ likes: { $gte: 500 } })
```

D) Embedded array

Agar comments me numeric field hota to:

JavaScript

```
// concept  
// db.posts.find({ "comments.score": { $gte: 5 } })
```

A) Single field

</> JavaScript

```
db.products.find({ price: { $lt: 30000 } })
```

</> JavaScript

```
db.courses.find({ fees: { $lt: 25000 } })
```

B) Array field

`tags` string array hai, so `$lt` practically useful nahi.

C) Embedded document field

likes < 200 वाले posts

❏ JavaScript

```
db.posts.find({ likes: { $lt: 200 } })
```

D) Embedded array

Conceptually:

❏ JavaScript

```
// db.posts.find({ "comments.score": { $lt: 3 } })
```

A) Single field

</> JavaScript

```
db.students.find({ age: { $lte: 20 } })
```

</> JavaScript

```
db.products.find({ rating: { $lte: 2 } })
```

B) Array field

Again, string arrays me `$lte` practical nahi.

C) Embedded document field

likes <= 100

⌘ JavaScript

```
db.posts.find({ likes: { $lte: 100 } })
```

D) Embedded array

Concept same:

⌘ JavaScript

```
// db.posts.find({ "comments.score": { $lte: 2 } })
```

7) Range query (single field)

Students fees between 50000 and 70000

⟨⟩ JavaScript

```
db.students.find({  
  fees: { $gte: 50000, $lte: 70000 }  
})
```

Employees salary between 60000 and 120000

⟨⟩ JavaScript

```
db.employees.find({  
  salary: { $gt: 60000, $lt: 120000 }  
})
```

8) Range on post likes

likes between 200 and 500

JavaScript

```
db.posts.find({  
  likes: { $gte: 200, $lte: 500 }  
})
```

9) Embedded array real examples from your posts

Rahul comment वाले posts

JavaScript

```
db.posts.find({ "comments.user": "Rahul" })
```



A) Students from Indore, Bhopal, Delhi

⌘ JavaScript

```
db.students.find({  
  city: { $in: ["Indore", "Bhopal", "Delhi"] }  
})
```

B) Students not from Indore, Bhopal

⌘ JavaScript

```
db.students.find({  
  city: { $nin: ["Indore", "Bhopal"] }  
})
```

C) Employees from IT or HR department

⟨/⟩ JavaScript

```
db.employees.find({  
  department: { $in: ["IT", "HR"] }  
})
```

D) Employees not in Sales/Admin

⟨/⟩ JavaScript

```
db.employees.find({  
  department: { $nin: ["Sales", "Admin"] }  
})
```

2) Array Field

Tumhare `posts` collection me:

JavaScript

```
tags: ["mongodb", "nodejs", "database"]
```

A) Posts jinke tags me mongodb ya nodejs ho

JavaScript

```
db.posts.find({  
  tags: { $in: ["mongodb", "nodejs"] }  
})
```

Meaning: agar `tags` array me inme se koi bhi ek value present hai, document match hoga.

B) Posts jinke tags me `java` ya `python` na ho

</> JavaScript

```
db.posts.find({
  tags: { $nin: ["java", "python"] }
})
```

3) Embedded / Nested Field

Tumhare `posts.comments` me:

</> JavaScript

```
comments: [
  { user: "Rahul", msg: "Nice" },
  { user: "Priya", msg: "Good" }
]
```



A) Jis post me comment user Rahul ya Priya ho

❏ JavaScript

```
db.posts.find({  
  "comments.user": { $in: ["Rahul", "Priya"] }  
})
```

B) Jis post me comment msg Nice ya Good ho

❏ JavaScript

```
db.posts.find({  
  "comments.msg": { $in: ["Nice", "Good"] }  
})
```

780ef2b4e4f

C) Jis post me comment user Amit ya Tarun na ho

</> JavaScript

```
db.posts.find({  
  "comments.user": { $nin: ["Amit", "Tarun"] }  
})
```

4) Embedded Array / Array of Embedded Documents

`comments` embedded array hai, aur uske andar `user`, `msg` fields hain.

A) Rahul / Priya comment वाले posts

</> JavaScript

```
db.posts.find({  
  "comments.user": { $in: ["Rahul", "Priya"] }  
})
```



5) Date + \$in

Agar exact multiple dates match karni ho:

JavaScript

```
db.orders.find({
  orderDate: {
    $in: [
      new Date("2024-01-02"),
      new Date("2024-02-03")
    ]
  }
})
```

7) Combined with other conditions

IT/HR employees from Indore

JavaScript

```
db.employees.find({
  department: { $in: ["IT", "HR"] },
  city: "Indore"
})
```

Rating 4/5 products from Delhi or Mumbai

JavaScript

```
db.products.find({
  rating: { $in: [4, 5] },
  city: { $in: ["Delhi", "Mumbai"] }
})
```

</> JavaScript

```
db.collectionName.find({
  $and: [
    { condition1 },
    { condition2 }
  ]
})
```

Single field example

Indore ke students jinki age 22 ho

</> JavaScript

```
db.students.find({
  $and: [
    { city: "Indore" },
    { age: 22 }
  ]
})
```

Employees jinka department IT ho aur salary > 100000

JavaScript

```
db.employees.find({
  $and: [
    { department: "IT" },
    { salary: { $gt: 100000 } }
  ]
})
```

Array field example

`posts.tags` array hai.

Jis post me mongodb tag ho aur likes > 300 ho

JavaScript

```
db.posts.find({
  $and: [
    { tags: "mongodb" },
    { likes: { $gt: 300 } }
  ]
})
```

Embedded array example

Rahul comment aur likes > 200 वाले posts

`</>` JavaScript

```
db.posts.find({
  $and: [
    { "comments.user": "Rahul" },
    { likes: { $gt: 200 } }
  ]
})
```

Students jo Indore ya Bhopal se ho

JavaScript

```
db.students.find({
  $or: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
})
```

Products jinki rating 5 ho ya price < 20000 ho

JavaScript

```
db.products.find({
  $or: [
    { rating: 5 },
    { price: { $lt: 20000 } }
  ]
})
```

Embedded array example

Rahul ya Priya comment वाले posts

`</>` JavaScript

```
db.posts.find({
  $or: [
    { "comments.user": "Rahul" },
    { "comments.user": "Priya" }
  ]
})
```

⌞ JavaScript

```
db.collectionName.find({
  fieldName: { $not: { operator } }
})
```

Single field example

Students jinki age 22 nahi ho

⌞ JavaScript

```
db.students.find({
  age: { $not: { $eq: 22 } }
})
```

Products jinka rating 5 nahi ho

❏ JavaScript

```
db.products.find({  
  rating: { $not: { $eq: 5 } }  
})
```

Employees jinki salary 100000 se zyada nahi ho

❏ JavaScript

```
db.employees.find({  
  salary: { $not: { $gt: 100000 } }  
})
```

Jis post me mongodb tag na ho

</> JavaScript

```
db.posts.find({  
  tags: { $not: { $eq: "mongodb" } }  
})
```

Embedded array example

Jis post me Rahul comment na ho

</> JavaScript

```
db.posts.find({  
  "comments.user": { $not: { $eq: "Rahul" } }  
})
```

4) \$nor

Meaning

Di hui **saari conditions false** honi chahiye.

Matlab **none of the conditions should be true.**

Syntax

</> JavaScript

```
db.collectionName.find({
  $nor: [
    { condition1 },
    { condition2 }
  ]
})
```

Students jo na Indore ke ho na Bhopal ke

JavaScript

```
db.students.find({
  $nor: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
})
```

Products jinki rating 5 bhi na ho aur price < 20000 bhi na ho

JavaScript

```
db.products.find({
  $nor: [
    { rating: 5 },
    { price: { $lt: 20000 } }
  ]
})
```



Embedded array example

Jis post me na Rahul comment ho na Priya comment ho

JavaScript

```
db.posts.find({
  $nor: [
    { "comments.user": "Rahul" },
    { "comments.user": "Priya" }
  ]
})
```

```
use companyDB

// AND
db.students.find({ $and: [ { city: "Indore" }, { age: 22 } ] })
db.posts.find({ $and: [ { tags: "mongodb" }, { likes: { $gt: 300 } } ] })

// OR
db.students.find({ $or: [ { city: "Indore" }, { city: "Bhopal" } ] })
db.posts.find({ $or: [ { "comments.user": "Rahul" }, { "comments.user": "Priya" } ] })

// NOT
db.students.find({ age: { $not: { $eq: 22 } } })
db.posts.find({ tags: { $not: { $eq: "mongodb" } } })

// NOR
db.students.find({ $nor: [ { city: "Indore" }, { city: "Bhopal" } ] })
db.posts.find({ $nor: [ { tags: "mongodb" }, { tags: "nodejs" } ] })
```

XOR Example 1

Students: city = Indore XOR age = 22

Matlab:

- ya to city Indore ho
- ya age 22 ho
- **but dono saath me na ho**

JavaScript

```
db.students.find({
  $or: [
    { city: "Indore", age: { $ne: 22 } },
    { city: { $ne: "Indore" }, age: 22 }
  ]
})
```

XOR Example 2

Products: rating = 5 XOR city = Delhi

JavaScript

```
db.products.find({
  $or: [
    { rating: 5, city: { $ne: "Delhi" } },
    { rating: { $ne: 5 }, city: "Delhi" }
  ]
})
```

XOR Example 3

Posts: tag mongodb XOR likes > 300

JavaScript

```
db.posts.find({
  $or: [
    { tags: "mongodb", likes: { $lte: 300 } },
    { tags: { $ne: "mongodb" }, likes: { $gt: 300 } }
  ]
})
```

XOR Example 4

Posts: Rahul comment XOR likes > 400

JavaScript

```
db.posts.find({
  $or: [
    { "comments.user": "Rahul", likes: { $lte: 400 } },
    { "comments.user": { $ne: "Rahul" }, likes: { $gt: 400 } }
  ]
})
```

MongoDB XNOR Examples

XNOR Example 1

Students: city = Indore XNOR age = 22

Matlab:

- dono true
ya
- dono false

JavaScript

```
db.students.find({
  $or: [
    { city: "Indore", age: 22 },
    { city: { $ne: "Indore" }, age: { $ne: 22 } }
  ]
})
```

XNOR Example 2

Products: rating = 5 XNOR city = Delhi

JavaScript

```
db.products.find({
  $or: [
    { rating: 5, city: "Delhi" },
    { rating: { $ne: 5 }, city: { $ne: "Delhi" } }
  ]
})
```

XNOR Example 3

Posts: Rahul comment XNOR likes > 400

JavaScript

```
db.posts.find({
  $or: [
    { "comments.user": "Rahul", likes: { $gt: 400 } },
    { "comments.user": { $ne: "Rahul" }, likes: { $lte: 400 } }
  ]
})
```

```
use companyDB
```

```
// XOR
```

```
db.students.find({  
  $or: [  
    { city: "Indore", age: { $ne: 22 } },  
    { city: { $ne: "Indore" }, age: 22 }  
  ]  
})
```

```
// XNOR
```

```
db.students.find({  
  $or: [  
    { city: "Indore", age: 22 },  
    { city: { $ne: "Indore" }, age: { $ne: 22 } }  
  ]  
})
```